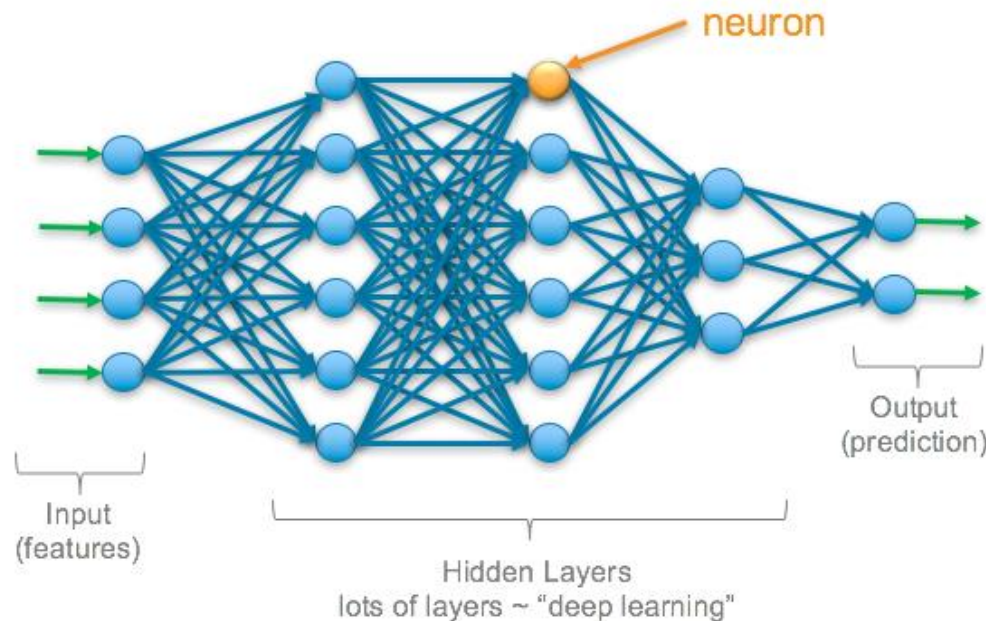


What is an Artificial Neural Network?

Neural networks can learn complex patterns using layers of **neurons** which mathematically transform the data

The layers between the input and output are referred to as “hidden layers”

A neural network can learn relationships between the features that other algorithms cannot easily discover



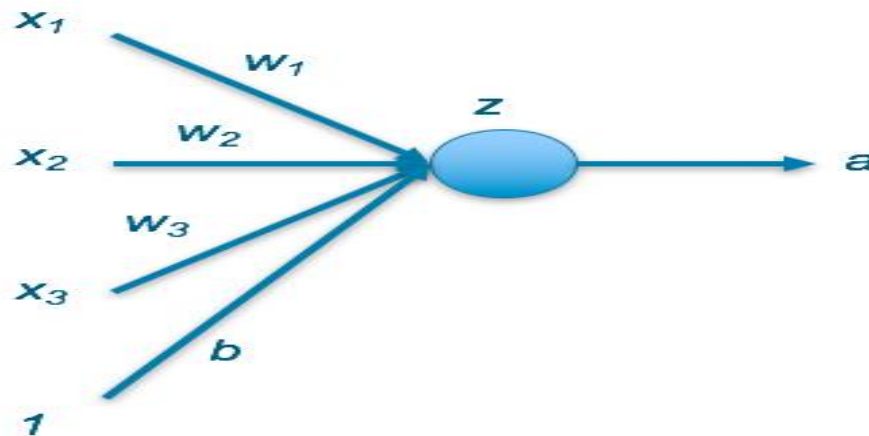
Multilayer Perceptron (MLP)

The above diagram is a ***Multilayer Perceptron (MLP)***. An MLP must have at least three layers: the input layer, a hidden layer and the output layer. They are fully connected; each node in one layer connects with a weight to every node in the next layer.

The term “deep learning” is coined for machine learning models built with many hidden layers: deep neural networks.

What is a neuron?

An artificial neuron (also referred to as a perceptron) is a mathematical function. It takes one or more inputs that are multiplied by values called “weights” and added together. This value is then passed to a non-linear function, known as an ***activation function***, to become the neuron’s output.



- The $\mathbf{x}$ values refer to inputs, either the original features or inputs from a previous hidden layer
- At each layer, there is also a bias (bias refers to a type of error that arises from simplifying assumptions made by a model, leading to systematic underestimation or overestimation of the true value. Essentially, it's the difference between the average prediction of a model and the actual value.) $\mathbf{b}$ which can help better fit the data
- The neuron passes the value $\mathbf{a}$ to all neurons it is connected to in the next layer, or returns it as the final value

The calculation starts with a linear equation:

$$\mathbf{z} = \mathbf{x}_1\mathbf{w}_1 + \mathbf{x}_2\mathbf{w}_2 + \mathbf{x}_3\mathbf{w}_3 + \mathbf{b}$$

Before adding a non-linear activation function:

$$\mathbf{a} = f(\mathbf{z})$$

What is an Activation Function?

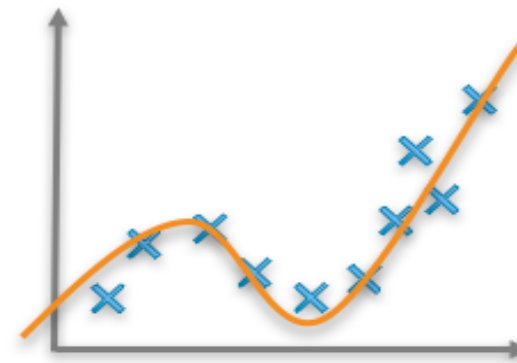
An activation function is a non-linear function applied by a neuron to introduce non-linear properties in the network.

A relationship is linear if a change in the first variable corresponds to a constant change in the second variable. A non-linear relationship means that a change in the first variable doesn't necessarily correspond with a constant change in the second. However, they may impact each other but it appears to be unpredictable.

A quick visual example, by introducing non-linearity we can better capture the patterns in this data



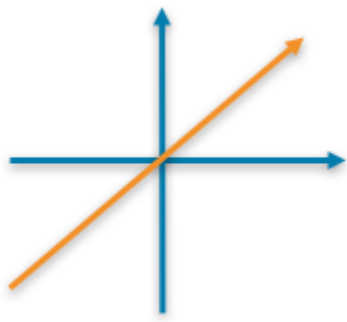
Linear function



Non-linear function

Best fit linear and non-linear models

Linear Activation Function



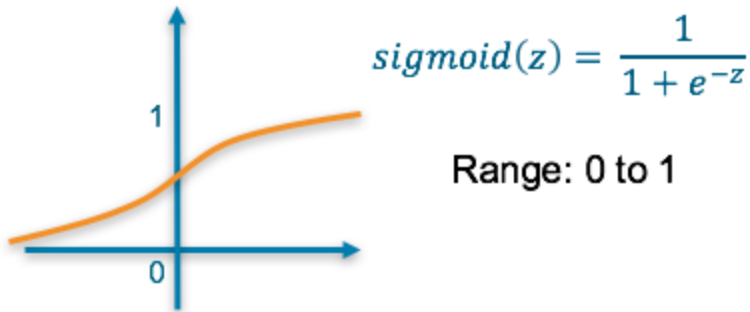
$$f(z) = az$$

Range: -infinity to infinity

- A straight line function: ***a*** is a constant value

- Values can get very large
- The linear function alone doesn't capture complex patterns

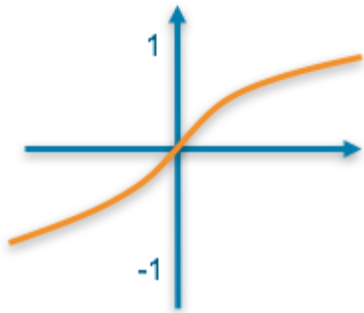
Sigmoid Activation Function



- A non-linear function so can capture more complex patterns

- Output values are bounded so don't get too large
- Can suffer from “vanishing gradient”

Hyperbolic Tangent Activation Function

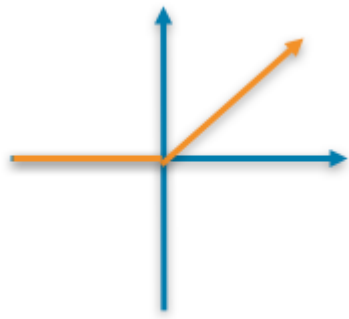


$$\tanh(z) = \frac{e^z - e^{-z}}{e^z + e^{-z}}$$

Range: -1 to 1

- A non-linear function so can capture more complex patterns
- Output values are bounded so don't get too large
- Can suffer from “vanishing gradient”

Rectified Linear Unit (ReLU) Activation Function



$$\text{relu}(z) = \max(0, z)$$

Range: 0 to infinity

- A non-linear function so can capture more complex patterns
- Values can get very large
- As it does not allow for negative values, certain patterns may not be captured
- Gradient can go towards 0 so weights are not updated: “dying ReLU problem”

Alternatively, instead of using 0.01, that can also be a parameter, α , which is then learned during training alongside the weights. This is referred to as Parametric ReLU (PReLU):

$$\text{prerelu}(z) = \begin{cases} \alpha z & \text{for } z < 0 \\ z & \text{for } z \geq 0 \end{cases}$$

Mike



Mohan



Sakib



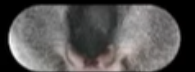
$$\text{face} = \text{eyes} * 0.2 + \text{nose} * 0.5 + \text{ears} * 0.3$$



Mike

 = 0.8

Mohan

 = 0.9

Sakib

 = 0.6

Serena

$$\begin{aligned}\text{face} &= \text{eyes} * 0.2 + \text{nose} * 0.5 + \text{ears} * 0.33 \\ &= 0.8 * 0.2 + 0.9 * 0.5 + 0.6 * 0.3\end{aligned}$$

Backward Error Propagation

